DataAgent — Architecture

Overview

DataAgent is an enterprise-grade intelligent data analysis platform built with a **frontend-backend separated B/S architecture**. The backend is implemented in Go as a single binary, with Worker and Scheduler running as goroutines within the same process. Asynchronous tasks are coordinated via Redis Stream.

The platform provides two modes of interaction:

- **Chat Mode**: Real-time conversational data queries with streaming responses (SSE)
- **Agent Mode**: Batch analysis tasks supporting both synchronous execution and asynchronous queue-based processing

A central **Agent Service** serves as the single entry point for all user requests. It manages Sessions, orchestrates the ADK Agent Engine for LLM inference and tool calling, and coordinates with Skills (SQL executor, statistics engine, knowledge base search, etc.).

System Diagram

```
Client → API Gateway (JWT + RBAC + Rate Limit + Security Filter)
         → Agent Service (Session + ADK Agent Engine)
           → [LLM Router → Skills (SQL/Stats/KB/Email/...)]
         → Security Filter (Output) → Audit Log → Response
```

Tech Stack

Layer	Technology	Version
Backend Language	Go	1.22+
Agent Framework	google.golang.org/adk	latest
Business Database	MongoDB	7.0+
Vector Database	Milvus	2.4+
Object Storage	SeaweedFS	RELEASE.2024+
Cache & Message Queue	Redis	7.2+
Secrets Management	HashiCorp Vault	1.18+
Memory System	Mem0	latest
Frontend	React / Next.js	18 / 14
Feishu SDK	go-lark/lark	latest

Services

Service	Description
`agent-service`	Central coordinator: Chat sync, Agent sync/async, Session management
`agent-engine`	ADK-based Agent engine: LLM routing, tool calling, security audit
`worker-pool`	Goroutine pool consuming Redis Stream for async task execution
`scheduler`	Cron-based scheduled task trigger (writes to Redis Stream)
`admin-service`	Admin dashboard APIs (users, roles, models, tasks, audit)
`im-gateway`	Feishu bot integration (Webhook + message routing)
`hermes-service`	Independent forwarding layer for free exploration mode

Data Stores

Store	Purpose
MongoDB	All business entities (users, sessions, tasks, reports, artifacts, audit logs)
Milvus	Vector embeddings for knowledge base semantic search
SeaweedFS	Binary files (session workspace, artifact content)
Redis	Query cache, message queue (Stream), session data
Mem0	Multi-granularity memory management
Vault	Secrets encryption (API keys, DB passwords, JWT secret)

Key Design Decisions

1. **Single Binary Deployment**: Reduces operational complexity for MVP. Worker and Scheduler run as goroutines, not separate services.
2. **Agent Service as Single Entry**: Chat and Agent modes share the same engine, avoiding code duplication.
3. **Redis Stream for Async**: Lightweight message queue without additional infrastructure (no RabbitMQ/Kafka).
4. **LLM-based Document Chunking**: Uses the main LLM for semantic chunk boundary detection instead of a dedicated embedding model.
5. **SQL AST Security**: Parse SQL with `pingcap/tidb/parser` to block write operations at the syntax level, not relying on LLM prompt constraints.

Directory Structure

```
data-agent/
├── cmd/server/main.go        # Entry point (single binary)
├── internal/
│   ├── api/                  # HTTP layer (handlers, middleware, router)
│   ├── logic/                # Shared business logic (Skill + Handler + Service)
│   │   ├── sql/              # SQL generation + AST validation
│   │   ├── stats/            # Statistics (regression, PCA, clustering)
│   │   ├── knowledge/        # Hybrid search + RRF ranking
```

```
│   │   ├── report/           # Markdown AST report validation
│   │   └── openapi/          # OpenAPI 3.0 parser
│   ├── service/              # Business services (chat/agent/admin/im)
│   ├── domain/               # Domain models, agent engine, skill registry, security audit
│   ├── worker/               # Async task worker pool
│   ├── queue/                # Redis Stream task queue + dead letter
│   ├── scheduler/            # Cron scheduler
│   ├── infra/                # Infrastructure (MongoDB/Milvus/Redis/SeaweedFS)
│   └── config/               # Configuration management
├── skills/                   # Skill implementations
├── frontend/                 # React/Next.js frontend
├── configs/                  # YAML config files
├── docs/                     # Public documentation
├── .agent/                   # AI agent instructions (SSOT)
├── docker-compose.yml
└── Makefile
```

References

- Product Requirements (PRD)
- Technical RFC
- Development Roadmap
- .agent/memory/ARCHITECTURE.md — detailed architecture for AI agents